[Trung Nguyen] Socket Programming - TCP/UDP Sockets & I/O Multiplexing

Socket Programming on Linux

Part 1. Overview

1.1. Protocol Independence

A protocol is a set of rules that two parties agree on in order to exchange data, for example IPv4, IPv6, TCP, UDP.

A program is called **protocol-dependent** when its source code is tightly bound to a specific protocol. For example, a client program written for IPv4 would use:

```
struct sockaddr_in servaddr;          // IPv4-specific address structure
servaddr.sin_family = AF_INET;
servaddr.sin_port   = htons(13);
inet_pton(AF_INET, "192.168.1.10", &servaddr.sin_addr);
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

To switch to IPv6, you have to change several things:

IPv4	IPv6
`sockaddr_in`	`sockaddr_in6`

IPv4	IPv6
`AF_INET`	`AF_INET6`
`sin_port`	`sin6_port`
`sin_addr`	`sin6_addr`

Every protocol change means another round of source code edits.

Protocol independence is a way of writing programs so that the source code is not hard-wired to any protocol and works with both IPv4 and IPv6 without modification. The standard approach is to use the `getaddrinfo()` function. It takes a hostname or an address string and returns the appropriate address structure (IPv4 or IPv6) together with the corresponding protocol family (domain) and socket type. The program simply uses these returned values to call `socket()` and `connect()`, without needing to know which protocol lies underneath.

This is possible because the sockets API was designed from the start to support many different protocol families. Functions such as `bind()` and `connect()` take a generic `struct sockaddr *` argument, while the concrete address structure varies by protocol.

1.2. The OSI Model

The OSI model (Open Systems Interconnection), published by ISO, divides network communication into 7 layers. Each layer has its own responsibility and only interacts with the layer directly above and directly below it.

OSI Layer	Internet Protocol Suite Equivalent	Where It Lives
7. Application	Application (HTTP, FTP, your program…)	User process
6. Presentation	↑ (merged into the application layer)	↑
5. Session	↑ (merged into the application layer)	↑
4. Transport	TCP, UDP	Kernel
3. Network	IPv4, IPv6	Kernel
2. Data link	Device drivers and hardware (e.g. Ethernet)	Device driver + hardware
1. Physical	↑	↑

Data encapsulation and de-encapsulation diagram

How is data sent over a network? Why does the OSI model need so many layers?

The diagram above shows how data is encapsulated and de-encapsulated as it travels across a network.

1. **Step 1:** When device A sends data to device B over the network using HTTP, an HTTP header is first added to the data at the application layer.
2. **Step 2:** Next, a TCP or UDP header is added. The data is packaged into TCP segments at the transport layer. This header contains the source port, destination port, and sequence number.
3. **Step 3:** The segments are then encapsulated with an IP header at the network layer. The IP header contains the source and destination IP addresses.
4. **Step 4:** The IP datagram gets a MAC header at the data link layer, containing the source and destination MAC addresses.
5. **Step 5:** The encapsulated frames are passed down to the physical layer and transmitted across the network as binary bits.
6. **Steps 6–10:** When device B receives the bits from the network, it performs de-encapsulation, the reverse of encapsulation. The headers are stripped off layer by layer, and finally device B can read the data.

> Networks are layered because each layer focuses only on its own responsibility. Each layer relies on its header to know how to process the data, and does not need to understand the meaning of data coming from other layers.

1.3. BSD Networking History

BSD (Berkeley Software Distribution) is a branch of the UNIX operating system developed by the CSRG (Computer Systems Research Group) at the University of California, Berkeley.

Key milestones:

Year	Release	Significance
1983	4.2BSD	The first widely distributed release with the sockets API and the TCP/IP protocol suite. This is where the sockets API was born.
1986–1990	4.3BSD (1986), 4.3BSD Tahoe (1988), 4.3BSD Reno (1990)	Continued improvements to the networking code.
1989, 1991	Net/1, Net/2	The networking code released separately as freely available source, without requiring an AT&T UNIX license.
1993–1995	4.4BSD (1993), 4.4BSD-Lite / Net/3 (1994), 4.4BSD-Lite2 (1995)	The final releases from Berkeley.

Part 2. Socket Addresses and Byte Ordering

2.1. Socket Address Structures

- A transport layer protocol in the TCP/IP suite needs both an IP address and a port number at each end to establish a connection. The combination of an IP address and a port number is called a **socket address**.
- The client socket address uniquely identifies the client process, just as the server socket address uniquely identifies the server process, as shown in the figure.

Client and server socket addresses

- To use the transport layer services on the Internet, we need a pair of socket addresses: the client socket address and the server socket address.
- These four pieces of information are carried in the network layer header and the transport layer header. The first header contains the IP addresses; the second contains the port numbers.

The new problem: there is only one set of socket functions, but they must accept many kinds of addresses.

The solution: every address structure follows one convention: the first field always indicates the address family, along with the length of the structure.

The kernel receives a block of bytes, reads the first field to know whether "this is IPv4" or "this is IPv6", and only then knows how to interpret the remaining bytes. The generic structure (`sockaddr`) is merely a "frame" expressing this convention; it is not used to hold real addresses. This is exactly what allows the sockets API to achieve protocol independence.

2.2. Value-Result Arguments

The process and the kernel live in two separate memory regions: **user space** (the process's memory) and **kernel space** (the kernel's memory). A process is not allowed to read kernel memory. So when the kernel wants to return an address (e.g. "what is the address of the client that just connected?"), it cannot hand the process a pointer into kernel memory. The only way: the process prepares a memory buffer in advance, and the kernel copies the data into it.

This raises two questions:

1. **How many bytes is the kernel allowed to write?** If the kernel writes more than the buffer the process provided, data spills into other memory (buffer overflow).
2. **How many bytes did the kernel actually write?** IPv4 and IPv6 addresses have different sizes, so the process needs to know the actual length of the result.

The solution: use a single length variable for both roles, and have the process pass the address of that variable so the kernel can modify it:

When	The length variable indicates
On input	Size of the buffer
On return	Number of bytes actually written

Socket address structure passed from the kernel to the process:

Socket address structure passed from kernel to process

2.3. Byte Ordering

Byte ordering illustration

Little Endian

Intel x86 processors store a 2-byte integer as follows: the least significant byte (low byte) comes first, followed by the most significant byte (high byte). This is called **little-endian** byte order.

Big Endian

In **big-endian** byte order, the most significant byte is stored at the lowest memory address, and the least significant byte at the highest memory address. Older PowerPC and Motorola 68k architectures typically use big-endian. Big-endian is also used in network communication and file storage.

Byte order matters when data is transferred between systems, or processed by systems with different byte orders. It must be handled correctly so that data is interpreted consistently across systems.

The byte order used by a machine's CPU is called the **host byte order**. Within a single machine there is no problem, since the CPU writes and reads using the same convention.

The real problem appears when transmitting over a network

A network only carries a sequence of bytes; it carries no information about "is this the high byte or the low byte". If a little-endian sender sends `90 1F`, a big-endian receiver will read it as `0x901F` =

36895. The same byte sequence, but the two sides interpret it as two different numbers.

The solution

Network protocols agree on a common convention: every numeric field in packet headers uses big-endian. This convention is called **network byte order**.

This gives rise to the **byte ordering functions**: they convert numbers from host byte order to network byte order when sending, and back when receiving. In essence:

- On a little-endian machine: they swap the byte order.
- On a big-endian machine: they do nothing, since host order already matches network order.

As a result, the same code runs correctly on any CPU, and the programmer does not need to know which kind their machine is. This is **portability**: source code that runs on many platforms without modification.

Part 3. The `inet_aton`, `inet_pton`, `inet_ntop` Functions

An IP address exists in two forms:

- **Presentation (text) form:** a character string humans can read and type, e.g. `"192.168.1.10"` (IPv4, called dotted-decimal notation: 4 decimal numbers separated by dots) or `"fe80::1"` (IPv6, hexadecimal groups separated by colons).
- **Numeric (binary) form:** the numeric value used by the kernel and by IP packet headers. For IPv4 it is a 32-bit number, for IPv6 it is 128 bits, in network byte order. This is the form stored in the address field of a socket address structure.

The two forms are completely different in memory. Users enter strings (from config files, the command line), while address structures need binary numbers. When the kernel returns a client address, you receive a binary number, but to print it to a log you need a string. Hence the need for functions that convert between the two forms.

3.1. `inet_aton` – string → binary (IPv4 only)

```
int inet_aton(const char *str, struct in_addr *addr);
```

- Converts the string `str` into a binary number and stores it in `addr`.
- Returns `1` if the string is valid, `0` if it is not.
- This function is quite "lenient": besides the standard 4-part form, it also accepts shorthand forms and hexadecimal/octal numbers (e.g. `"10.1"` or `"0x7f.1"`), which can sometimes be surprising.

3.2. `inet_pton` – string → binary (both IPv4 and IPv6)

```
int inet_pton(int family, const char *str, void *addr);
```

Parameter	Meaning
`family`	`AF_INET` or `AF_INET6`, tells the function how to interpret the string
`str`	The address string
`addr`	Where the result is stored. For IPv4, a pointer to `struct in_addr`; for IPv6, a pointer to `struct in6_addr`. The `void *` type allows both

The return value has three cases, which is an improvement over `inet_addr`:

Value	Meaning
`1`	Success
`0`	The string is not a valid format for the chosen address family
`-1`	Error, e.g. unsupported address family (`errno = EAFNOSUPPORT`)

> With `AF_INET`, this function is strict: it only accepts the standard 4-part decimal form, not the shorthand or hexadecimal forms that `inet_aton` accepts.

3.3. `inet_ntop` – binary → string

```
const char *inet_ntop(int family, const void *addr, char *str, socklen_t size);
```

Parameter	Meaning
`addr`	Pointer to the binary address (`in_addr` or `in6_addr`)

Parameter	Meaning
`str`	Buffer you provide for the function to write the string into
`size`	Size of that buffer, so the function does not overflow it. If the buffer is too small, the function returns `NULL` and sets `errno = ENOSPC`

Returns: the pointer `str` on success, `NULL` on error.

Part 4. Elementary Socket Functions

Functions covered: `socket`, `connect`, `bind`, `listen`, `accept`.

The call sequence for a TCP client/server pair is as follows:

```
        SERVER                                    CLIENT
   socket()                                  socket()
      |                                         |
   bind()      ← bind "well-known" IP:port     |  (usually no bind, kernel picks an ephemeral
port)
      |                                         |
   listen()    ← CLOSED → LISTEN               |
      |                                         |
   accept()    ← blocks, waits           connect()  ← sends SYN, 3-way handshake
      |  ◄──────────── 3-way handshake ──────────┤
      |  returns a NEW socket (connfd)          |  returns 0 once ESTABLISHED
   read/write  ◄──────────── data ──────────► write/read
   close()                                   close()
```

4.1. `socket()`: create a communication endpoint

```
#include <sys/socket.h>


int socket(int family, int type, int protocol);
```

Parameter	Meaning	Common Values
`family`	Protocol family (domain)	`AF_INET` (IPv4), `AF_INET6` (IPv6), `AF_LOCAL`/`AF_UNIX` (local sockets within the machine), `AF_ROUTE`, `AF_KEY`
`type`	Socket type	`SOCK_STREAM` (byte stream), `SOCK_DGRAM` (datagram), `SOCK_SEQPACKET`, `SOCK_RAW`
`protocol`	Specific protocol	`0` to let the kernel choose the default protocol for the family/type pair, or `IPPROTO_TCP`, `IPPROTO_UDP`, `IPPROTO_SCTP`

Common valid combinations:

	`AF_INET`	`AF_INET6`
`SOCK_STREAM`	TCP	TCP
`SOCK_DGRAM`	UDP	UDP
`SOCK_RAW`	Raw IPv4	Raw IPv6

Return value: a non-negative integer called a socket descriptor (`sockfd`), similar to a file descriptor. Returns `-1` on error and sets `errno`.

What it really does:

- `socket()` only creates a data structure in the kernel and specifies which protocol the socket will use. At this point the socket **has no address** (no local IP and port), and is **not connected** to anyone.
- Since a socket is also a descriptor, you can later use `read()`, `write()`, `close()` on it just like a file.
- Combined with `getaddrinfo()` (Part 1), we pass `ai_family`, `ai_socktype`, `ai_protocol` directly to `socket()`, so the code is protocol-independent.

4.2. `connect()`: the client establishes a connection to the server

```
int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

Parameter	Meaning
`sockfd`	The socket returned by `socket()`

Parameter	Meaning
`servaddr`	Pointer to the server's socket address structure (IP + port). The generic type is `struct sockaddr *`, so a cast is required: `(struct sockaddr *) &servaddr`
`addrlen`	Size of that structure, e.g. `sizeof(servaddr)`. This is a value argument (from process to kernel), **not** a value-result argument

Returns: `0` on success, `-1` on error.

With TCP: `connect()` initiates the three-way handshake and returns only when the connection is established (`ESTABLISHED`) or an error occurs.

- The client does not need to call `bind()` first. If it has not bound, the kernel automatically chooses the source IP (based on the outgoing route) and an ephemeral port.
- The TCP state moves from `CLOSED` to `SYN_SENT`, then to `ESTABLISHED` upon receiving SYN+ACK.

Three classic errors:

Situation	Error	Explanation
No response to the SYN (retransmitted several times, about 75 seconds in total)	`ETIMEDOUT`	Server is down, IP does not exist on the LAN…
Server responds with RST	`ECONNREFUSED`	**Hard error**: the server host exists but no process is listening on that port
An intermediate router returns ICMP "destination unreachable"	`EHOSTUNREACH` / `ENETUNREACH`	**Soft error**: the kernel keeps retrying and only reports the error after about 75 seconds

> **Important:** if `connect()` fails, that socket can no longer be used. You must `close()` it and call `socket()` again. This is why a loop that tries each address returned by `getaddrinfo()` always creates a new socket for each attempt.

4.3. `bind()`: assign a local address to a socket

```
int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

Parameter	Meaning
`myaddr`	The local address (IP + port of this machine) to assign to the socket
`addrlen`	Size of the structure (value argument)

Returns: `0` on success, `-1` on error.

Why is it needed? A server must have a fixed, well-known port (e.g. 80 for HTTP, 13 for daytime) so clients know where to connect. Clients usually do not need it, since they just use an ephemeral port.

Four ways to combine IP and port when binding:

IP Address	Port	Result
Wildcard (`INADDR_ANY` / `in6addr_any`)	`0`	Kernel chooses both IP and port
Wildcard	non-zero	Kernel chooses IP, process chooses port. **This is the most common approach for servers**
Specific IP	`0`	Process chooses IP, kernel chooses port
Specific IP	non-zero	Process chooses both

- The wildcard address (`INADDR_ANY`) means "accept connections to any IP of this machine". If the machine has multiple network interfaces, the server accepts connections on all of them.
- With a specific IP, the server only accepts connections sent to that exact IP.
- If the kernel is left to choose the port (port = 0), `bind()` does not return the chosen port (since `myaddr` is `const`). To find out which port was chosen, call `getsockname()`.

Typical example:

```
struct sockaddr_in servaddr;
memset(&servaddr, 0, sizeof(servaddr));
servaddr.sin_family      = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);   // convert to network byte order (Part 2)
servaddr.sin_port        = htons(13);
bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
```

> **Common error:** `EADDRINUSE` ("Address already in use"). This usually happens when restarting a server right after shutting it down, while the old connection is still in the `TIME_WAIT` state. The fix is to set the `SO_REUSEADDR` option with

> `setsockopt()` before calling `bind()`. Also, binding to a port < 1024 on UNIX requires root privileges (`EACCES`).

4.4. `listen()`: put the socket into listening mode

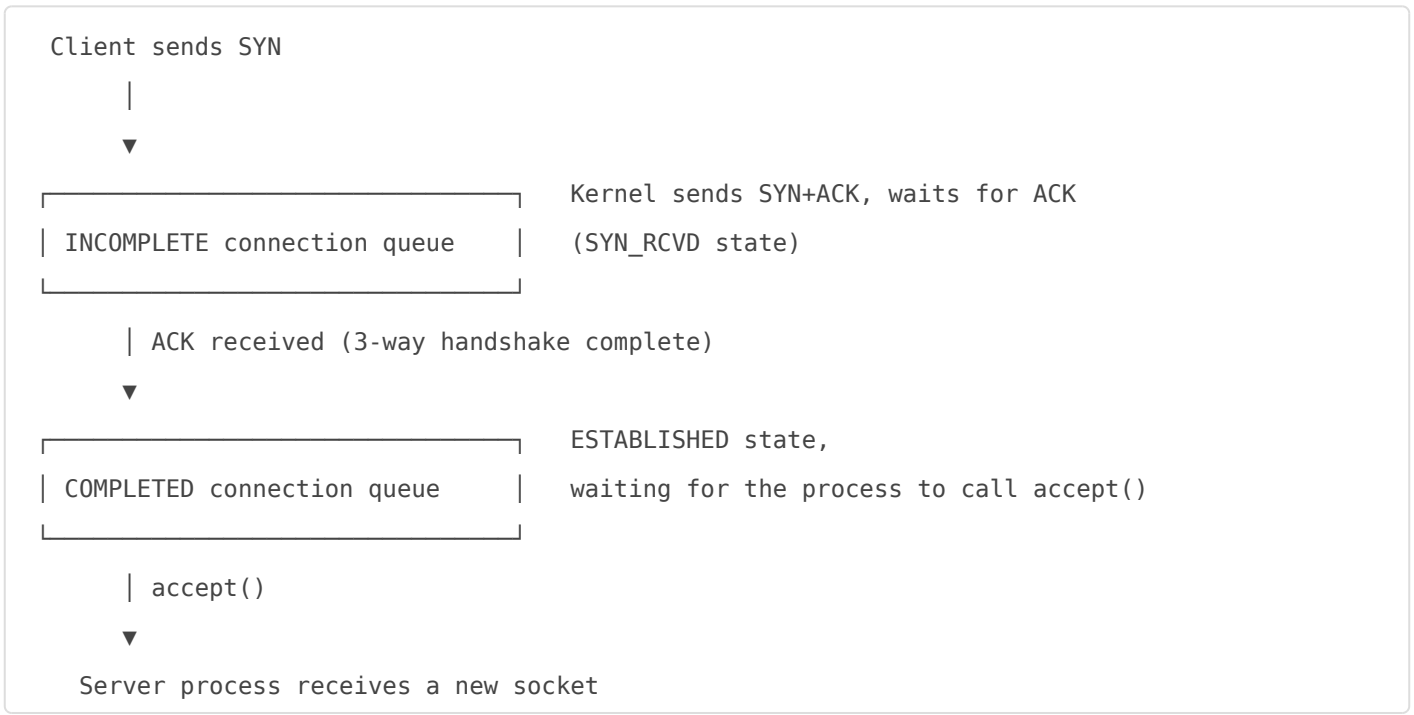
```
int listen(int sockfd, int backlog);
```

Returns: `0` on success, `-1` on error. Only servers call this function.

The function does two things:

1. **Turns an active socket into a passive socket.** By default, a socket created by `socket()` is assumed to be active, i.e. a client socket that will call `connect()`. `listen()` tells the kernel that this socket should accept incoming connections. The TCP state moves from `CLOSED` to `LISTEN`.
2. **Specifies the `backlog`:** the maximum number of connections the kernel will queue for this socket.

The two queues the kernel maintains for a listening socket:



- **Key point:** the three-way handshake is handled entirely by the kernel, regardless of whether the server is currently calling `accept()`. `accept()` only retrieves connections that have already completed.

- `backlog` (on modern Linux) is the limit of the completed queue. Its value is capped by `/proc/sys/net/core/somaxconn`. You can use the constant `SOMAXCONN`.

The two queues of a listening socket

- When the queue is full and a new SYN arrives, TCP **ignores** that SYN (it does not send an RST). The client will retransmit the SYN and may be accepted later. If an RST were sent, the client would wrongly conclude that "there is no server" (`ECONNREFUSED`).
- Do not set `backlog = 0`, since different systems interpret this value differently.

4.5. `accept()`: retrieve a completed connection from the queue

```
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

Parameter	Meaning
`sockfd`	The listening socket, already passed through `bind()` and `listen()`
`cliaddr`	Buffer provided by the process; the kernel writes the client's address into it
`addrlen`	Value-result argument (see Part 2.2)

For `addrlen`:

- On input: `*addrlen` = size of the `cliaddr` buffer.
- On return: `*addrlen` = number of bytes the kernel actually wrote.
- If you do not need the client's address, you can pass `NULL` for both `cliaddr` and `addrlen`.

Returns: a **NEW** socket descriptor, called the connected socket (`connfd`). Returns `-1` on error.

Distinguishing the two kinds of socket (the most commonly confused point):

	Listening socket (`listenfd`)	**Connected socket (`connfd`)**
Created by	`socket()` + `bind()` + `listen()`	`accept()`
How many	One for the server's entire lifetime	One per client
Used for	Only accepting new connections	`read()`/`write()` data with exactly one client
Closed when	The server shuts down	Done serving that client

- **Blocking behavior:** if the completed queue is empty, the process is put to sleep until a new connection arrives.

- Each TCP connection is identified by a 4-tuple (client IP, client port, server IP, server port). Therefore many `connfd`s can share the same server port (e.g. 13) without being confused with each other.

Part 5. Concurrent Servers

The problem: an iterative server serves one client at a time. If one client holds the connection for a long time, the other clients must wait in the accept queue.

The solution: after each `accept()`, call `fork()` to create a child process dedicated to that client. The parent process immediately returns to `accept()`.

5.1. `fork()`

```
pid_t fork(void);
```

The function is called **once** but returns **twice**:

Returns in	Value
Parent process	The child's PID (`> 0`)
Child process	`0`
Error	`-1`

The child is a copy of the parent and shares the sockets opened before `fork()`: both point to the same socket in the kernel.

5.2. Program Skeleton

```
for (;;) {
    connfd = accept(listenfd, ...);
    if (fork() == 0) {          // child
        close(listenfd);
        doit(connfd);
        close(connfd);
        exit(0);
    }
    close(connfd);              // parent
```

```
}
```

5.3. Why must `close()` be called on both sides?

Each socket has a **reference count**. `close()` only decrements the count by 1; the kernel only truly closes the socket (sends FIN) when the count reaches 0. After `fork()`, both `listenfd` and `connfd` have a reference count of 2.

Forgetting to close	Consequence
Parent does not `close(connfd)`	Descriptor leak (eventually `accept()` fails with `EMFILE`), and the client never receives a FIN
Child does not `close(listenfd)`	The child needlessly holds the listening port

5.4. Pros and Cons

Pros	Cons
Simple code, each child handles only one client	`fork()` is expensive
A crashing child does not affect other clients	One process per connection, hard to scale to large numbers

---

Part 6. The `close`, `getsockname`, `getpeername` Functions

6.1. `close()`: close a socket

```
#include <unistd.h>

int close(int sockfd);
```

Returns: `0` on success, `-1` on error.

With TCP:

- `close()` marks the descriptor as closed and returns immediately. From then on, the process may no longer use this descriptor for `read()`/`write()`.
- The kernel still tries to send any data remaining in the send buffer, then sends a FIN to begin the normal TCP connection termination.
- `close()` only decrements the reference count by 1. The kernel only sends FIN when the count reaches 0 (see Part 5.3).

When different behavior is needed:

Need	Use
Send FIN immediately, regardless of the reference count	`shutdown(sockfd, SHUT_WR)`
Half-close: stop sending but still able to read	`shutdown(sockfd, SHUT_WR)`
Make `close()` block until data is sent, or abort the connection with RST	The `SO_LINGER` option

6.2. `getsockname()` and `getpeername()`

```
#include <sys/socket.h>

int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
int getpeername(int sockfd, struct sockaddr *peeraddr,  socklen_t *addrlen);
```

Function	Returns which address
`getsockname()`	The local address (own IP + port) bound to the socket
`getpeername()`	The peer address (IP + port of the remote side)

- `addrlen` is a value-result argument, just like in `accept()`. On input it is the buffer size; on return it is the number of bytes the kernel wrote.
- **Returns:** `0` on success, `-1` on error. `getpeername()` on an unconnected socket fails with `ENOTCONN`.

When to use them:

Situation	Function	Reason
Client calls `connect()` without `bind()`	`getsockname()`	Find out the IP and ephemeral port the kernel chose
`bind()` with port 0	`getsockname()`	Find out which port the kernel chose (`bind()` does not return it)
Server binds the wildcard address (`INADDR_ANY`)	`getsockname(connfd, ...)` after `accept()`	Find out which of the machine's IPs the client connected to (on multi-homed hosts)

Situation	Function	Reason
The address family of a socket is unknown	`getsockname()` with `struct sockaddr_storage`	Read the `ss_family` field to tell whether it is `AF_INET` or `AF_INET6`
A child process created with `fork()` then `exec()`s another program (e.g. `inetd`)	`getpeername()`	After `exec()`, the new program only has the descriptor; the `cliaddr` variable filled by `accept()` is lost. The only option is to ask the kernel for the client address

Example: print the client's ephemeral port after connecting:

```
struct sockaddr_in local;
socklen_t len = sizeof(local);
getsockname(sockfd, (struct sockaddr *) &local, &len);
printf("Local port: %d\n", ntohs(local.sin_port));
```

Part 7. TCP Echo Server and Client (`str_echo` / `str_cli`)

TCP echo client–server model

This is a basic TCP client–server model: the client sends data to the server, and the server receives it and sends the same data back to the client.

Function	Runs on	Job
`str_cli()`	Client	Read a line from `stdin`, send it over the socket, read the server's reply and print it to `stdout`
`str_echo()`	Server (child process)	Read data from the connected socket and write it back to the client

This example shows the complete flow of a TCP connection:

1. The client calls `connect()`, the server accepts the connection with `accept()`.
2. The server `fork()`s a child process to run `str_echo()` (see Part 5).
3. Both sides exchange data with `read()` / `write()`.
4. TCP carries data as a **byte stream** between the two processes and does not preserve boundaries between writes, so the program must handle short reads/writes itself.

Part 8. Connection Termination, the `SIGPIPE` Signal and Crash Scenarios

8.1. Normal Termination

The TCP connection state machine:

TCP state machine

#	State	Meaning
1	`LISTEN`	Waiting for a connection request from any remote TCP host
2	`SYN-SENT`	Connection request (SYN) sent, waiting for a matching connection request from the peer
3	`SYN-RECEIVED`	Connection request both sent and received, waiting for the confirming ACK
4	`ESTABLISHED`	Connection open; received data is delivered to the application
5	`FIN-WAIT-1`	FIN sent, waiting for its ACK or for a FIN from the peer
6	`FIN-WAIT-2`	FIN acknowledged, waiting for a FIN from the peer
7	`CLOSE-WAIT`	FIN received from the peer, waiting for the local application to call `close()`
8	`CLOSING`	Both sides sent FIN simultaneously, waiting for the ACK of our FIN
9	`LAST-ACK`	FIN received and our own FIN sent, waiting for the final ACK
10	`TIME-WAIT`	Waiting long enough (2 × MSL) to be sure the peer received the final ACK
11	`CLOSED`	A fictional state representing no connection

Segment sequence during connection termination:

TCP connection termination sequence

8.2. The `SIGPIPE` Signal

SIGPIPE scenario

Sequence of events:

1. The client runs normally, sends `"hi there"`, and the server echoes it back.
2. The server's child process is killed. The server kernel sends FIN to the client.
3. The user types `"bye"` on the client.
4. The first `writen()` sends the first byte. This is allowed because the socket has only received FIN.
5. No process holds the socket on the server side anymore, so the server TCP responds with **RST**.
6. The client calls `writen()` a second time on a socket that has received RST.
7. The kernel sends `SIGPIPE` to the client process. The default action of `SIGPIPE` is to terminate the process, and the shell prints:

```
Broken pipe
```

Run output: Broken pipe

Cause:

- The **first** write makes the server send RST.
- The **second** write is what delivers `SIGPIPE` to the process.

> **Rule:** writing to a socket that has received a **FIN** is fine; writing to a socket that has received an **RST** is an error.

If the program ignores or catches `SIGPIPE`, `write()` returns `-1` with `errno = EPIPE` instead of killing the process.

8.3. Server/Client Crash Scenarios

Crash of Server Host

Scenario: the client and server have an open TCP connection. Suddenly the **whole server host** crashes (power loss, cable unplugged, kernel panic…), not just the server process.

Key point: a crashed host has no chance to send FIN or RST, so the client receives no notification at all.

Failure	Does the client find out immediately?	Why
Only the server **process** dies	Yes	The server kernel is still running, closes the socket itself and sends FIN
The whole server **host** crashes	No	Nothing is sent

What happens on the client side:

1. The client calls `write()` / `send()`. The call succeeds because the data has only been copied into the client kernel's send buffer.
2. The client TCP sends the segment, but no ACK comes back.
3. TCP retransmits several times, doubling the interval each time (exponential backoff).
4. Throughout this period, the client is blocked in `read()`.
5. When TCP gives up (about 9 minutes on BSD according to Stevens; on Linux typically 15 minutes or more, depending on the `tcp_retries2` parameter), `read()` returns an error:

Error	When
`ETIMEDOUT`	Timed out with no response at all
`EHOSTUNREACH` / `ENETUNREACH`	An intermediate router sent back an ICMP "destination unreachable" message

Server host crash

Detecting it early: if the client only reads and never sends, it will never notice that the server has crashed (see case 1b). Solutions:

- Set a timeout on `read()` (the `SO_RCVTIMEO` option, or use `select()` / `poll()`).
- Enable `SO_KEEPALIVE`: after a period of silence, TCP automatically sends probes to check whether the peer is still alive.
- Implement an application-level heartbeat.

Crashing and Rebooting of Server Host

1. The client and server have an open TCP connection and have exchanged data normally.
2. The server host crashes and reboots. The client has no idea, since a crashed host sends neither FIN nor RST.
3. Connection state lives in RAM, so after rebooting the server has lost all information about the old connection. As far as the server is concerned, that connection never existed.
4. The client sends a line of data, and the segment reaches the server.
5. The server TCP receives a segment for a connection it does not know, so it replies with **RST**.
6. The client, blocked in `read()`, receives the RST and `read()` returns the error `ECONNRESET`.

Case 1: `read()` when the peer loses connectivity

A common misconception: `read()` does not send data, so there is no waiting for an ACK. `read()` only waits for data from the peer. The outcome depends on whether our side still has sent data that has not been ACKed.

1a. We previously `write()` data that has not been ACKed:

- TCP keeps that data in the send buffer and retransmits it several times at increasing intervals.
- Meanwhile `read()` is blocked.
- When TCP runs out of retransmissions (Linux: `tcp_retries2`, default 15), the kernel flags an error on the socket.
- `read()` returns `-1` with `errno = ETIMEDOUT`.

1b. We only read and have not sent anything:

- There is no data to retransmit, so TCP sends nothing on the network and cannot detect anything wrong.
- `read()` blocks forever, with no error.
- This is exactly why you need `SO_KEEPALIVE`, a `select()` / `poll()` timeout, or an application-level heartbeat.

Case 2: `write()` when the peer has lost connectivity

Key point: `write()` returning success does **not** mean the peer received the data. `write()` just copies the data into the kernel's send buffer and returns immediately.

1. The first `write()` succeeds (returns the number of bytes written), even though the peer is unreachable.
2. TCP sends the data, gets no ACK, and retransmits several times.
3. If the program keeps calling `write()` until the send buffer fills up, `write()` starts to block (or returns `EAGAIN` if the socket is non-blocking).
4. When TCP gives up, the socket records the `ETIMEDOUT` error. The error is reported on the **next** `read()` / `write()` call, not on the `write()` that sent the data.
5. Once the socket has an error, the next `write()` raises `SIGPIPE` (which terminates the program by default). If `SIGPIPE` is ignored, `write()` returns the error `EPIPE`.

Part 9. UDP: `recvfrom` / `sendto`, UDP Echo Server/Client, Lost

Datagrams

9.1. `recvfrom()` and `sendto()`

UDP architecture:

UDP client–server model

The most important point: UDP has **no** `connect()` / `accept()` step like TCP. Instead, UDP uses two dedicated functions to send and receive data:

```
#include <sys/socket.h>

ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags,
                 struct sockaddr *from, socklen_t *addrlen);

ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t addrlen);
```

Returns: number of bytes read or written on success, `-1` on error.

	TCP	UDP
Client	`connect()` then `write()`	Sends a datagram directly with `sendto()`, including the server address
Server	`accept()` then `read()`	Calls `recvfrom()`, waits for a datagram from any client

`recvfrom()` returns both the **datagram** and the **protocol address** of the client that sent it, so the server knows where to send the reply.

The `to` argument of `sendto()`

`to` is a socket address structure containing the destination address (IP + port) the datagram is sent to. The size of the structure is passed in `addrlen`.

```
sendto(sockfd, buf, len, 0,
       (struct sockaddr *) &server_addr, sizeof(server_addr));
```

```
server_addr
    ├── IP address
    └── Port
```

The `from` argument of `recvfrom()`

`recvfrom()` fills the structure pointed to by `from` with the protocol address of the sender of the datagram.

```
struct sockaddr_in client_addr;
socklen_t addrlen = sizeof(client_addr);

recvfrom(sockfd, buf, sizeof(buf), 0,
         (struct sockaddr *) &client_addr, &addrlen);
```

After `recvfrom()` returns:

```
client_addr
    ├── Client IP
    └── Client port
```

- The server knows which client the datagram came from.
- `addrlen` is a value-result argument (see Part 2.2): on return it holds the number of bytes the kernel actually wrote into `client_addr`.

9.2. UDP Echo Server/Client

Architecture:

UDP echo model

No EOF: UDP is connectionless, so there is no concept of EOF as in TCP. With TCP, when the peer closes the connection, `read()` returns `0`. UDP has no connection to close, so the server cannot rely on EOF to exit its loop.

Iterative, not concurrent: there is no call to `fork()`; a single server process handles all clients.

Server type	Usually
TCP	Concurrent
UDP	Iterative

```
for (;;) {
    n = recvfrom(sockfd, buf, MAXLINE, 0, (struct sockaddr *) &cliaddr, &len);
    sendto(sockfd, buf, n, 0, (struct sockaddr *) &cliaddr, len);
}
```

```
Client A ──┐
           │
Client B ──┼──> UDP Server Process
           │
Client C ──┘
```

The server processes datagrams one at a time.

The UDP socket receive buffer

Each UDP socket has a **receive buffer** that acts as an implied queue. Every datagram arriving for the socket is placed in this buffer; each call to `recvfrom()` returns the next datagram in **FIFO** (First In, First Out) order.

When many datagrams arrive at once, you can increase the receive buffer size:

1. Read the current value with `getsockopt(..., SO_RCVBUF, ...)`.
2. Set a larger value with `setsockopt(..., SO_RCVBUF, ...)` (see Part 10.1).

The value must not exceed the system ceiling; if it does, the kernel caps it at the ceiling.

```
# Maximum allowed receive buffer size (system ceiling)
sysctl net.core.rmem_max

# Default receive buffer size for new sockets (if setsockopt is not called)
sysctl net.core.rmem_default
```

Checking rmem_max and rmem_default

9.3. Lost Datagrams

Lost UDP datagrams

UDP packet loss occurs when datagrams fail to reach their destination, leaving gaps in the transmitted data.

UDP has no mechanism to confirm that packets were received. Unlike TCP (which guarantees all data arrives and retransmits lost packets), UDP sends packets without waiting for an ACK. A lost packet is simply gone.

UDP packets can be lost in three places:

Where	Cause
On the way out	The application sends a very large number of packets. Each UDP socket has a send buffer, and the Linux kernel pushes packets out as fast as it can. If the network card is slow or overwhelmed and cannot keep up with the rate packets are queued, the queue overflows and packets are dropped
In transit	Network congestion, routing problems or other factors prevent the packet from reaching its destination
On the way in	The packet reaches the host and is placed in the socket's receive buffer. If the buffer is full or too small (the application cannot read fast enough), new packets are dropped

The receive buffer size determines how many packets can be held at once without loss. See how to check and adjust it in Part 9.2 and the `socket(7)` man page.

Part 10. Socket Options: `getsockopt` / `setsockopt`, `fcntl`

10.1. `getsockopt()` and `setsockopt()`

These two functions read and change the options of a socket.

```
#include <sys/socket.h>

int getsockopt(int sockfd, int level, int optname,
               void *optval, socklen_t *optlen);

int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
```

Function	Purpose
`getsockopt()`	Get the current value of an option
`setsockopt()`	Set a new value for an option

Returns: `0` on success, `-1` on error (and sets `errno`).

Parameters:

Parameter	Meaning
`sockfd`	Descriptor of the socket whose option is read/set
`level`	The layer the option belongs to: `SOL_SOCKET` (generic, for all sockets), `IPPROTO_IP` (IPv4), `IPPROTO_TCP` (TCP-specific)
`optname`	Option name, e.g. `SO_KEEPALIVE`, `SO_RCVTIMEO`
`optval`	Pointer to memory holding the option value. For `setsockopt()` it is the value to set; for `getsockopt()` it is where the kernel writes the result
`optlen`	Size of `optval`. `setsockopt()` takes a value; `getsockopt()` takes a pointer because the kernel writes back the actual size (value-result argument)

10.2. Common options

Level	Option	Meaning
`SOL_SOCKET`	`SO_REUSEADDR`	Allows `bind()` to an address/port that is in the `TIME_WAIT` state. Usually set on the server's listening socket, before `bind()`
`SOL_SOCKET`	`SO_KEEPALIVE`	Periodically sends probes to detect a dead peer (see case 1b)
`SOL_SOCKET`	`SO_RCVTIMEO` / `SO_SNDTIMEO`	Maximum wait time for `read()` / `write()`; on timeout returns `EAGAIN` / `EWOULDBLOCK`
`SOL_SOCKET`	`SO_RCVBUF` / `SO_SNDBUF`	Receive / send buffer size
`SOL_SOCKET`	`SO_ERROR`	Read-only (only with `getsockopt()`): fetch the pending error on the socket and clear it

10.3. Examples

Enable keepalive on a socket:

```
int on = 1;
if (setsockopt(sockfd, SOL_SOCKET, SO_KEEPALIVE, &on, sizeof(on)) < 0) {
    perror("setsockopt SO_KEEPALIVE");
}
```

Set a 5-second timeout on `read()`:

```
struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
```

Read the current receive buffer size:

```
int rcvbuf;
socklen_t len = sizeof(rcvbuf);
if (getsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &rcvbuf, &len) == 0) {
    printf("SO_RCVBUF = %d\n", rcvbuf);
}
```

10.4. `fcntl()`

`fcntl()` is a system call that reads or changes the attributes (file status flags) of a file descriptor. Since a socket is also a file descriptor, `fcntl()` is commonly used to change how a socket behaves.

```
#include <fcntl.h>


int fcntl(int fd, int cmd, ... /* int arg */);
```

Usage	Meaning
`F_GETFL`	Read the socket's current flags
`F_SETFL` + `O_NONBLOCK`	Put the socket into non-blocking mode
`F_SETFL` + `O_ASYNC`	Put the socket into signal-driven I/O mode
`F_SETOWN`	Set the process / process group that receives `SIGIO` / `SIGURG`

Example: put a TCP socket into non-blocking mode:

```
int flags = fcntl(sockfd, F_GETFL, 0);
fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);
```

You must read the old flags and OR in `O_NONBLOCK`; calling `F_SETFL` with just `O_NONBLOCK` would clear the other flags.

After that, `read(sockfd, buf, sizeof(buf))` no longer blocks indefinitely: if no data is available, it returns `-1` immediately with `errno = EAGAIN` / `EWOULDBLOCK`.

Revision #3
Created 2026-09-25 01:56:46 UTC by trungnguyen
Updated 2026-09-28 10:37:05 UTC by trungnguyen